Building an AI Observability Platform with OpenTelemetry

A technical architecture article on OpenTelemetry traces, Phoenix integration, AI reliability metrics, dashboards, and platform observability.

Introduction

AI applications are different from traditional APIs. A normal backend request usually has a clear control flow: receive input, validate it, call a database or service, return a response. An AI request often contains several additional layers: retrieval, prompt construction, model generation, token accounting, grounding checks, quality evaluation, and failure handling.

From the perspective of an AI Data Platform Architect, this changes the observability problem.

It is not enough to know whether an endpoint returned `200 OK`. We need to understand where the request spent time, how many tokens were used, whether retrieval found useful context, whether the model failed, whether the response was grounded, and whether quality is trending in the right direction.

The `05-ai-observability-platform` project explores that problem by building an instrumented AI API with FastAPI, OpenTelemetry traces, Arize Phoenix integration, local metrics storage, and a lightweight observability dashboard.

The goal is to make AI behavior measurable.

Why AI Observability Matters

AI systems introduce new sources of uncertainty.

A model call can be slow. A prompt can become too large. Retrieval can return weak context. A response can be fluent but unsupported. A request can succeed from an HTTP perspective while still failing from a user-value or reliability perspective.

Traditional API monitoring usually focuses on metrics like:

- request count
- latency
- error rate
- CPU and memory
- database performance

Those metrics still matter, but AI platforms need additional signals:

- model latency
- retrieval latency
- prompt token count
- completion token count

- total token usage
- retrieval hit rate
- source count
- response quality score
- model or generation errors
- trace context across the full AI workflow

This project treats those signals as first-class platform metrics.

Project Overview

The project implements a small AI API that simulates an observable RAG-style workflow. The API exposes a single instrumented AI endpoint:

```
POST /ask
```

When a request arrives, the system runs an observed pipeline:

```
POST /ask
-> ai.request
-> ai.retrieval
-> ai.model.generate
-> ai.quality.evaluate
```

Each stage records timing and AI-specific attributes. The API also stores request-level metrics in a local SQLite database and exposes metrics through API endpoints and a local dashboard.

The stack includes:

```
| Layer | Technology |
| ---------------- | ---------------------------------------- |
| API | FastAPI + Uvicorn |
| Tracing | OpenTelemetry |
| Trace viewer | Arize Phoenix via OTLP |
| Local metrics | SQLite |
| Dashboard | Built-in HTML dashboard + dashboard spec |
| Reliability docs | Markdown report |
```

This is intentionally lightweight, but the architecture mirrors the observability patterns needed in production AI systems.

The Instrumented AI Workflow

The core of the project is an observed `/ask` workflow.

At a high level, the request does four things:

1. Receives a user question.
2. Retrieves relevant context from a small local knowledge base.
3. Generates a mock answer from the retrieved context.
4. Scores a simple quality signal based on source support.

The important part is not the mock generation itself. The important part is how the system observes each stage.

The API records:

- total request latency
- retrieval latency
- model generation latency
- prompt tokens
- completion tokens
- total tokens
- retrieval hit status
- source count
- quality score
- error status and exception details

This creates a measurement framework that can later be connected to real RAG retrieval, real model serving, and real quality evaluation.

OpenTelemetry Trace Design

OpenTelemetry gives the platform a way to represent the AI request as a trace instead of a single opaque API call.

The project defines a span map:

```
| Span | Purpose |
| ---------------------- | ------------------------------------ |
| `POST /ask` | FastAPI request span |
| `ai.request` | Root span for the AI workflow |
| `ai.retrieval` | Measures retrieval work |
| `ai.model.generate` | Measures model generation work |
| `ai.quality.evaluate` | Measures response quality evaluation |
```

This structure matters because AI latency is rarely one thing.

If a user says the application is slow, the platform needs to answer:

- Was the API slow?
- Was retrieval slow?
- Was the model slow?
- Did token count spike?
- Did quality evaluation add overhead?
- Did an exception occur inside a model or retrieval step?

By separating the workflow into spans, the platform can identify the actual source of latency.

AI-Specific Trace Attributes

The project adds AI-specific attributes to spans:

- `ai.question.length`

- `ai.model.name`
- `ai.request.latency_ms`
- `ai.retrieval.latency_ms`
- `ai.model.latency_ms`
- `ai.tokens.prompt`
- `ai.tokens.completion`
- `ai.tokens.total`
- `ai.retrieval.hit`
- `ai.quality.score`

These attributes turn traces into operational evidence.

For example, if model latency increases, the trace can show whether that increase correlates with larger prompts, larger completions, or a specific model name. If answer quality drops, the trace can show whether retrieval returned sources at all.

From an AI Data Platform Architect's perspective, these attributes are the beginning of an AI control plane. They connect system behavior to model behavior.

Phoenix Integration

The project supports exporting traces to Arize Phoenix using OTLP.

The local flow is:

```
FastAPI app
-> OpenTelemetry spans
-> OTLP gRPC exporter
-> Phoenix collector on :4317
-> Phoenix UI on :6006
```

The Docker Compose setup runs the API and Phoenix together. In that mode, the API exports traces to:

```
http://phoenix:4317
```

Phoenix provides a visual trace waterfall. That is useful because many AI failures are easier to understand visually. A slow request may show a long model generation span. A poor answer may show retrieval with too few sources. A failed request may show exactly which span recorded an exception.

The project also supports a console exporter for simple local testing:

```
OTEL_EXPORTER=console uvicorn api.app.main:app --reload --port 8003
```

This makes it easy to inspect raw spans without running the full Phoenix stack.

Local Metrics Store

Traces are excellent for investigating individual requests, but platform teams also need aggregate metrics.

This project stores request-level metrics in SQLite. Each `/ask` request records:

- timestamp

- route
- status
- model name
- request latency
- retrieval latency
- model latency
- prompt tokens
- completion tokens
- total tokens
- retrieval hit
- quality score
- error message

The API exposes two metrics endpoints:

```
GET /metrics/summary
GET /metrics/events
```

`/metrics/summary` returns aggregate reliability metrics such as:

- total requests
- error count
- error rate
- average request latency
- average retrieval latency
- average model latency
- average total tokens
- retrieval hit rate
- average quality score

`/metrics/events` returns recent request-level events.

In a production system, this local SQLite store would likely be replaced by Prometheus, OTLP metrics, a warehouse-backed event table, or another operational data store. But the shape of the data is the key architectural point.

Dashboard Design

The project includes a lightweight local dashboard at:

```
http://localhost:8003/dashboard
```

The dashboard summarizes:

- total requests

- error rate
- average request latency
- average retrieval latency
- average model latency
- average token usage
- retrieval hit rate
- average quality score
- recent request events

The project also includes a dashboard specification file:

```
dashboards/ai-observability-dashboard.json
```

That dashboard spec defines panels for:

```
| Panel | Why It Matters |
| ----------------- | ------------------------------------ |
| Request latency | User-facing performance |
| Model latency | LLM runtime bottlenecks |
| Retrieval latency | Vector store or retrieval bottlenecks |
| Token usage | Cost and context pressure |
| Error rate | Reliability baseline |
| Retrieval hit rate | RAG grounding signal |
| Quality score | Response usefulness and source support |
```

From an architecture perspective, the dashboard is not just a visualization. It is an operating model. It defines what the platform team believes is important enough to monitor.

Reliability Goals

The project's reliability report defines several goals:

```
| Goal | Signal |
| ---------------------- | ---------------------------------------------------- |
| Fast enough for users | Request latency and model latency stay within target |
| Grounded answers | Retrieval hit rate and source count remain high |
| Predictable model usage | Token counts do not spike unexpectedly |
| Clear failures | Errors are captured with trace context |
| Measurable quality | Quality score trends are visible over time |
```

These goals reflect a broader truth: AI reliability is not only uptime.

An AI system can be available but slow. It can be fast but ungrounded. It can return responses but use too many tokens. It can avoid errors but produce low-quality answers. Observability needs to cover all of those dimensions.

Suggested SLOs

The project proposes initial service-level objectives:

```
| SLO | Initial Target |
| -------------------- | ------------------------------------------------ |
| `/ask` availability | 99% successful responses locally during test runs |
| p95 request latency | Under 2 seconds for mock pipeline |
| p95 retrieval latency | Under 250 ms for local retrieval |
| p95 model latency | Under 1.5 seconds for local model calls |
```

```
| Retrieval hit rate | Above 90% on known-answer questions |
| Error rate | Below 1% during steady-state runs |
```

These are local lab targets, not production commitments. But they demonstrate an important practice: AI platforms need explicit expectations.

Without targets, every system is either “working” or “broken” based on anecdotal feedback. With SLOs, the platform can measure whether it is meeting reliability and quality expectations.

Failure Modes

The reliability report identifies several important AI failure modes:

```
| Failure | Detection Signal | Response |
| ------------------- | -------------------------- |
---------------------------------------------------- |
| Model runtime slow | High `ai.model.latency_ms` | Reduce model size, enable streaming,
scale serving |
| Retrieval degraded | Low retrieval hit rate | Reindex documents, tune chunking, improve
embeddings |
| Prompt too large | Token count spike | Add context compression, lower retrieved chunk
count |
| Model exception | Error span and error metric | Retry safe failures, surface fallback
response |
| Poor answer quality | Quality score drops | Run eval set, inspect prompt/context, add
guardrails |
```

This is where AI observability becomes different from traditional API observability.

A platform team needs to know not only whether the service is up, but whether the AI workflow is behaving responsibly. Retrieval can degrade. Token usage can become expensive. Prompts can grow too large. Quality can drop silently. Model calls can fail in ways that need trace context.

Why This Matters for AI Data Platforms

AI data platforms sit at the intersection of data systems, model systems, and application systems.

They need to support:

- retrieval-augmented generation
- embedding workflows
- model-serving APIs
- evaluation pipelines
- prompt templates
- token budgets
- source grounding
- observability and governance
- user-facing reliability

Observability is the connective tissue across those layers.

For an AI Data Platform Architect, the core question is not simply “Did the request finish?” The better question is:

Did the system retrieve the right context, generate a timely answer, stay within token budget, avoid errors, and produce a response that was useful and grounded?

That question requires traces, metrics, dashboards, and reliability goals designed specifically for AI workloads.

Production Improvements

This project is a local lab, but it points toward several production improvements:

- replace SQLite metrics with Prometheus, OTLP metrics, or warehouse-backed event tables
- add p50, p95, and p99 latency aggregation
- trace real RAG calls from the Phase 4 document assistant
- trace real model calls from the Phase 3 local LLM serving lab
- add prompt template version and retrieval index version attributes
- track model name, embedding model, and vector store configuration
- add quality evaluation datasets instead of a local heuristic
- add alerting for error rate, token spikes, latency outliers, and retrieval degradation
- connect dashboards to Grafana or managed observability tooling
- add distributed trace propagation across API, worker, retrieval, and model services

These improvements would turn the local lab into a production-grade AI observability layer.

Final Takeaway

The `05-ai-observability-platform` project demonstrates how to observe an AI API as a workflow, not just as an HTTP endpoint.

It instruments an AI request with OpenTelemetry spans, exports traces to Phoenix, records request-level metrics, exposes metrics endpoints, renders a dashboard, and documents reliability goals for AI systems.

The most important lesson is that AI observability must measure both system behavior and model behavior.

Latency, errors, and uptime still matter. But for AI platforms, they are not enough. The platform also needs visibility into retrieval, token usage, model latency, source grounding, quality signals, and failure modes.

That is the foundation for operating reliable AI data platforms.